Activity 1: Representing Java Object in JSON

1. **Create a Java class named `Student` with with a parameterized constructor, getters, and setters for its attributes:**

- **Name (String)**
- **ID (int)**
- **Major (String)**
- **GPA (double)**

Solution:

```
public class Student {
  private String name;
  private int id;
  private String major;
  private double gpa;

  // Parameterized constructor to initialize all attributes
  public Student(String name, int id, String major, double gpa) {
    this.name = name;
    this.id = id;
    this.major = major;
    this.gpa = gpa;
  }

  // Getters for all attributes
  public String getName() {
    return name;
  }

  public int getId() {
    return id;
  }
```

```
  public String getMajor() {
    return major;
  }

  public double getGpa() {
    return gpa;
  }

  // Setters for all attributes (optional)
  public void setName(String name) {
    this.name = name;
  }

  public void setId(int id) {
    this.id = id;
  }

  public void setMajor(String major) {
    this.major = major;
  }

  public void setGpa(double gpa) {
    this.gpa = gpa;
  }
}
```

2. Write a single JSON object to represent the data of Student object in notepad

Solution :

```
{
  "name": "Alice",
  "id": 12345,
  "major": "Computer Science",
  "gpa": 3.8
}
```

3. Write Array of Student Objects in JSON

Solution :

```
[
 {
  "name": "Alice",
  "id": 12345,
  "major": "Computer Science",
  "gpa": 3.8
 },
 {
  "name": "Bob",
  "id": 54321,
  "major": "Mathematics",
  "gpa": 3.9
 },
 {
  "name": "Charlie",
  "id": 98765,
  "major": "Physics",
  "gpa": 3.7
 }
]
```

Activity 2 : Representing Java Object in XML

1. **Represent a single student object in XML as**

   - **The root element is `<student>`, which defines the main container for the student information.**
   - **Each attribute of the student is represented by a child element within the `<student>` tag.**

Solution :

```
<student>
 <name>Alice</name>
 <id>12345</id>
 <major>Computer Science</major>
 <gpa>3.8</gpa>
</student>
```

2. **Represent a single student object in XML as XML attributes instead of child elements. (e.g., `id="12345"`)**

Solution :

```
<student name="Alice" id="12345" major="Computer Science" gpa="3.8">
</student>
```

Activity 3 : Design a REST API Endpoint

1. **Creating Resource**

Scenario: You're building a web application for managing a to-do list. You want to create a REST API endpoint to allow users to add new tasks to their lists.

Task: Design a REST API endpoint following the RESTful principles:

- **URL:** Define the URL path for the endpoint.
- **HTTP Method:** Specify the appropriate HTTP method for adding tasks.
- **Request Body:** Describe the data format (e.g., JSON) and structure of the request body containing the new task information.
- **Response:** Define the format and content of the response sent back by the API.

RESTful Considerations:

- Use nouns for resources in the URL path (e.g., /tasks).
- Use appropriate HTTP methods for CRUD operations:
- Follow a consistent data format like JSON for request and response bodies.
- Provide clear and informative status codes in the response (e.g., 201 Created for successful task creation).

Solution:

URL: `/api/tasks`

HTTP Method: `POST` (This indicates creating a new resource)

Request Body (JSON):

JSON

```
{
  "title": "Buy groceries",
  "description": "Milk, bread, eggs"
}
```

- `title`: (String) Required - The title of the new task.

- `description`: (String) Optional - A detailed description of the task.

Response:

Success (Status Code: 201 Created):

JSON

```
{
  "id": 1,  // Unique identifier for the newly created task
  "title": "Buy groceries",
  "description": "Milk, bread, eggs",
}
```

- The response includes the newly created task information with an assigned ID and potentially the creation timestamp.

Error (Example: Status Code: 400 Bad Request):

JSON

```
{
  "error": "Missing required field: title"
}
```

- The response provides an error message indicating the reason for the failed request (e.g., missing required fields).

2. Reading single Resource

Scenario: You want to retrieve existing tasks from the to-do list using a REST API endpoint.

Task: Design an endpoint to allow users to access their tasks.

Design a REST API endpoint following the RESTful principles:

- **URL:** Define the URL path for the endpoint.
- **HTTP Method:** Specify the appropriate HTTP method for adding tasks.
- **Request Body:** Describe the data format (e.g., JSON) and structure of the request body containing the new task information.
- **Response:** Define the format and content of the response sent back by the API.

RESTful Considerations:

- Use nouns for resources in the URL path (e.g., /tasks).
- Use appropriate HTTP methods for CRUD operations:
- Follow a consistent data format like JSON for request and response bodies.
- Provide clear and informative status codes in the response (e.g., 201 Created for successful task creation).

Solution:

URL: `/api/tasks` (Same as add task endpoint, following RESTful resource naming)
HTTP Method: `GET` (This indicates retrieving data)

Response:

Success (Status Code: 200 OK):

JSON

```
[
  {
    "id": 1,
    "title": "Buy groceries",
    "description": "Milk, bread, eggs"
    "
  },
```

```
  {
    "id": 2,
    "title": "Finish report",
    "description": "Due tomorrow"
  }
]
```

- The response is an array containing JSON objects representing individual tasks.
- Each task object includes its ID, title, description

Error (Example: Status Code: 404 Not Found):

JSON

```
{
  "error": "No tasks found"
}
```

- The response provides an error message if no tasks are found for the user or based on the provided filters.